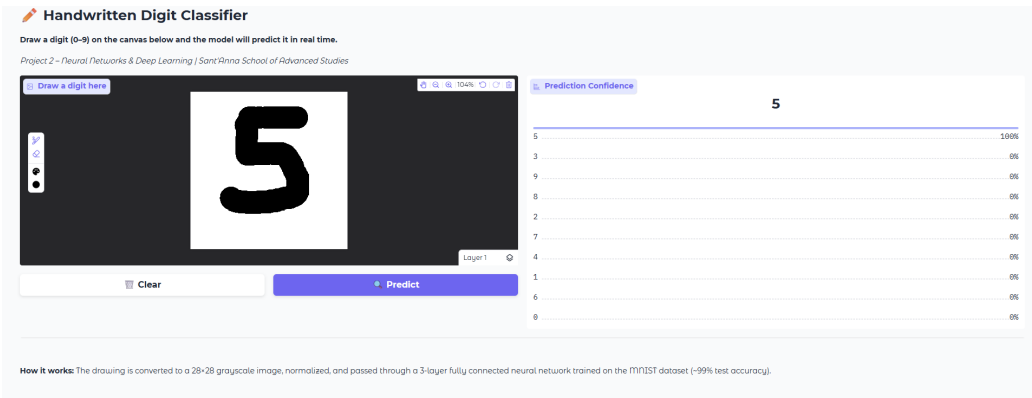
University of Pisa, Pisa, Italy
Sant'Anna School of Advanced Studies, Pisa, Italy

Neural Networks & Deep Learning Theoretical Foundations

Handwritten Digit Classifier

A Real-Time 3-Layer Neural Network on MNIST



Author: Kabir Bakhshaei

Student ID: 1312090810

Email: kabir.bakhshaei@santannapisa.it

Program: Ph.D. in Artificial Intelligence

Institutions: Sant'Anna School of Advanced Studies & University of Pisa

Lab: SMARTLab, BioRobotics Institute

Supervisor: Prof. Giovanni Stabile

Course Professor: Prof. Giorgio Buttazzo

Delivery Date: April 1, 2026

Contents

1 Introduction and Application Description **2**

2 Neural Network Architecture **2**
2.1 Layer Description 2
2.2 Forward Pass 2
2.3 Key Design Choices 2

3 Implementation Details **3**
3.1 Hardware and Software Environment 3
3.2 Dataset 3
3.3 Dataset Augmentation 3
3.4 Programming Language and Framework 3

4 Hyperparameters **4**

5 Evaluation Metrics **4**

6 Experiments and Results **4**
6.1 Training Curves 4
6.2 Test Set Performance 5

7 Graphical Interface **6**

8 Conclusion **6**

References **7**

Introduction and Application Description

Handwritten digit recognition is one of the most fundamental problems in machine learning and computer vision. This project designs, trains, and deploys a fully connected neural network that classifies handwritten digits (0–9) drawn by a user in real time via a graphical interface.

The application is built around the **MNIST dataset** [1], one of the most widely used benchmarks in pattern recognition. The system takes a 28×28 grayscale image drawn by the user as input and produces a probability distribution over 10 digit classes. The user interacts through a browser-based drawing canvas built with **Gradio** [6], and the predicted digit is shown instantly alongside confidence scores for all 10 classes.

This type of application is directly relevant to postal code recognition, bank cheque processing, and form digitization.

Neural Network Architecture

Layer Description

The network is a 3-layer fully connected (MLP) architecture. Table 1 summarises each layer.

Layer	Input	Output	Activation
Flatten	28×28	784	—
FC1	784	512	ReLU
Dropout	—	—	$p = 0.3$
FC2	512	256	ReLU
Dropout	—	—	$p = 0.3$
FC3 (Output)	256	10	Softmax

Table 1: Layer-by-layer description of the DigitClassifier.

The exact number of trainable parameters is: $(784\times512 + 512) + (512\times256 + 256) + (256\times10 + 10) = 535{,}818$.

Forward Pass

Let $\mathbf{x} \in \mathbb{R}^{784}$ denote the flattened input. The forward pass computes:

$$\mathbf{h}^{(1)} = \mathrm{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \quad \mathbf{W}^{(1)} \in \mathbb{R}^{512\times784} \tag{1}$$

$$\mathbf{h}^{(2)} = \mathrm{ReLU}(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}), \quad \mathbf{W}^{(2)} \in \mathbb{R}^{256\times512} \tag{2}$$

$$\mathbf{z} = \mathbf{W}^{(3)}\mathbf{h}^{(2)} + \mathbf{b}^{(3)}, \quad \mathbf{W}^{(3)} \in \mathbb{R}^{10\times256} \tag{3}$$

$$\hat{\mathbf{p}} = \mathrm{Softmax}(\mathbf{z}) \in \mathbb{R}^{10} \tag{4}$$

Key Design Choices

ReLU activation [5] is used after each hidden layer: $f(x) = \max(0, x)$, avoiding the vanishing gradient problem.

Dropout ($p = 0.3$) [3] randomly deactivates 30% of neurons per forward pass during training, preventing overfitting. During inference all neurons are active.

Xavier uniform initialisation [4] sets weights from $\mathcal{U}(-a, a)$ where $a = \sqrt{6/(\text{fan_in} + \text{fan_out})}$, maintaining stable gradient flow from the start.

Cross-Entropy loss is minimised during training:

$$\mathcal{L}_{\text{CE}} = -\sum_{c=0}^{9} y_c \log \hat{p}_c = -\log \hat{p}_{c^*} \tag{5}$$

where c^* is the index of the ground-truth digit.

Implementation Details

Hardware and Software Environment

Component	Specification
CPU	Intel Core i7-12650H
GPU	NVIDIA RTX 3060 Laptop (6 GB VRAM)
RAM	16 GB
CUDA Toolkit	12.1
Python	3.10.19
PyTorch	2.5.1 (CUDA 12.1, cuDNN 9)
TorchVision	0.20.1
Gradio	6.10.0
Scikit-learn	1.7.2

Table 2: Hardware and software environment used for training.

Dataset

The **MNIST dataset** [1] consists of 70,000 grayscale 28×28 images of handwritten digits. A 10% validation split was carved from the training set:

- Training: 54,000 images
- Validation: 6,000 images
- Test: 10,000 images

Images are normalised: $\hat{x} = (x - 0.1307)/0.3081$.

Dataset Augmentation

Applied to the training set only: random rotation (±10ř) and random affine translation (±10%), improving robustness to different handwriting styles.

Programming Language and Framework

The project is implemented in **Python 3.10.19** using **PyTorch 2.5.1**. The codebase follows a modular MLOps-style structure with all hyperparameters centralised in `config.yaml`:

- `src/model.py` — network architecture
- `src/train.py` — training loop with early stopping
- `src/evaluate.py` — evaluation and metrics
- `src/utils.py` — preprocessing and utilities
- `app.py` — Gradio web interface
- `config.yaml` — all hyperparameters

Hyperparameters

Hyperparameter	Value
Batch size	64
Learning rate (initial)	0.001
Max epochs	50 (early stopping active)
Optimizer	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Loss function	Cross-Entropy
Dropout probability	0.3
Weight initialisation	Xavier uniform
LR scheduler	ReduceLROnPlateau (patience=3, factor=0.5, min_lr=10^{-5})
Early stopping patience	7 epochs, $\delta = 0.0001$
Validation split	10%

Table 3: Hyperparameter configuration used for training.

The **Adam optimiser** [2] adapts the learning rate per weight using momentum $\mathbf{m}_t$ and velocity $\mathbf{v}_t$ estimates. **Early stopping** monitors validation loss and halts training if no improvement greater than $\delta = 0.0001$ is seen for 7 consecutive epochs.

Evaluation Metrics

The following metrics are computed on the test set using `scikit-learn`. For each class c, let TP_c, FP_c, FN_c be the true positives, false positives, and false negatives:

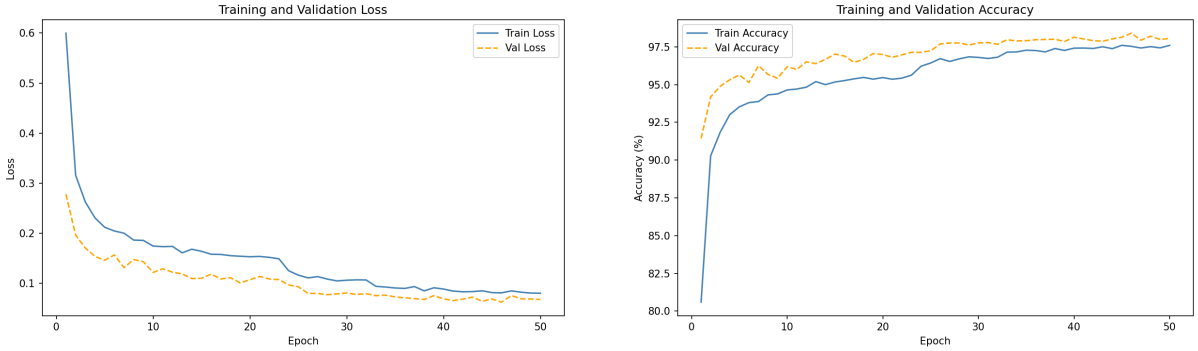
$$\mathrm{Accuracy} = \frac{\sum_c \mathrm{TP}_c}{N} \qquad \mathrm{Precision}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c} \tag{6}$$

$$\mathrm{Recall}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c} \qquad \mathrm{F1}_c = 2 \cdot \frac{\mathrm{Precision}_c \cdot \mathrm{Recall}_c}{\mathrm{Precision}_c + \mathrm{Recall}_c} \tag{7}$$

Experiments and Results

Training Curves

Training ran for the full 50 epochs. The best validation loss was 0.0626 at epoch 50, with a best validation accuracy of 98.4%. Training loss decreased from $\sim$0.60 at epoch 1 to 0.0803 at epoch 50. The validation loss remained consistently below the training loss throughout — an expected effect of Dropout being disabled at validation time.



(a) Loss curves over 50 epochs. **(b)** Accuracy curves over 50 epochs.

Figure 1: Training and validation loss and accuracy. Validation consistently exceeds training due to Dropout being disabled at evaluation time.

Test Set Performance

The model achieves a **test accuracy of 98.98%** on the full 10,000-sample MNIST test set. Table 4 reports per-class metrics. The hardest digits are 7 (F1= 0.9834) and 9 (F1= 0.9831), reflecting visual similarity with other digits.

Digit	Precision	Recall	F1	Support
0	0.9909	0.9959	0.9934	980
1	0.9921	0.9991	0.9956	1135
2	0.9904	0.9961	0.9932	1032
3	0.9901	0.9931	0.9916	1010
4	0.9868	0.9919	0.9893	982
5	0.9921	0.9865	0.9893	892
6	0.9916	0.9875	0.9895	958
7	0.9844	0.9825	0.9834	1028
8	0.9928	0.9846	0.9887	974
9	0.9870	0.9792	0.9831	1009
Macro Avg	**0.9898**	**0.9896**	**0.9897**	**10000**

Table 4: Per-class classification report on the MNIST test set.

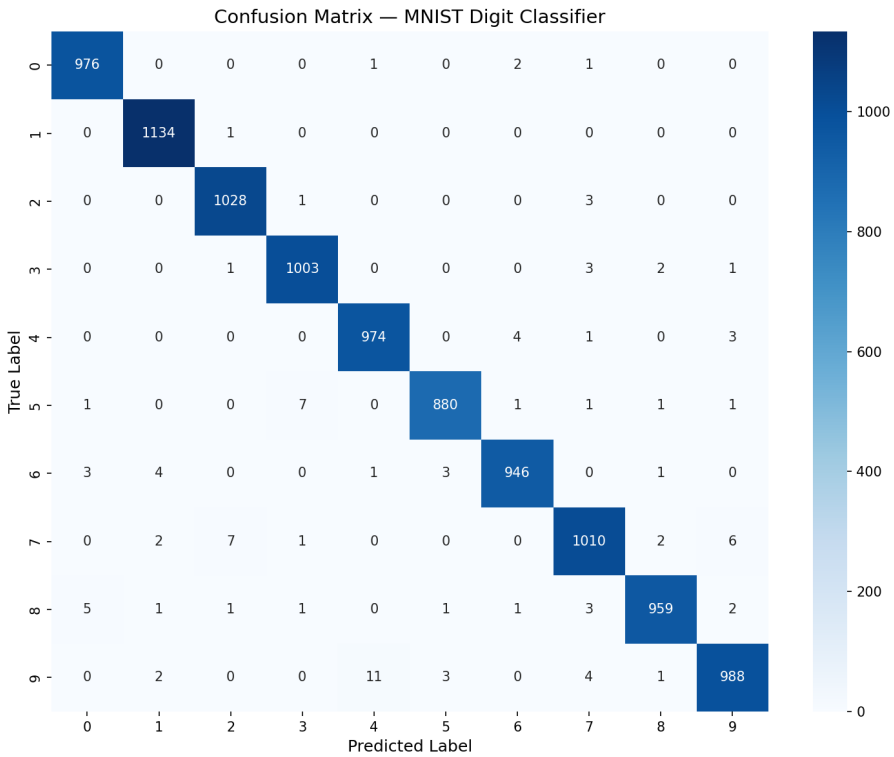


Figure 2: Confusion matrix on the MNIST test set. The diagonal is strongly dominant. The most frequent misclassification is digit 9 predicted as 4 (11 cases).

Graphical Interface

The user interface is implemented using **Gradio 6.10.0** [6] and runs in any standard web browser at `http://localhost:7860`. It consists of a drawing canvas (280×280 px, black brush on white background), a label output showing the predicted class and confidence bar chart for all 10 digit classes, and Clear/Predict buttons.

The canvas input is preprocessed before inference: converted from RGB to grayscale, inverted to match MNIST format (white digit on black background), resized to 28×28 using Lanczos resampling, and normalised. If the model confidence falls below 0.6, a low-confidence warning is displayed.

Conclusion

This project demonstrates the design, training, and deployment of a 3-layer fully connected neural network for real-time handwritten digit classification. The network consists of 535,818 trainable parameters and was trained on MNIST for 50 epochs using the Adam optimiser, Xavier initialisation, Dropout regularisation, and early stopping (which did not trigger, indicating stable convergence).

The model achieves a **test accuracy of 98.98%** with a macro-averaged F1 of 0.9897. The best validation loss was 0.0626 with a corresponding validation accuracy of 98.4%. The model runs interactively through a Gradio interface where users draw digits and receive real-time predictions with per-class confidence scores.

The modular MLOps-style code structure with centralised `config.yaml` makes every hyperparameter traceable and reproducible. A detailed extended report (28 pages) with worked examples, mathematical derivations, and supplementary diagrams is included alongside this document.

References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, *Gradient-based learning applied to document recognition*, Proceedings of the IEEE, 86(11):2278–2324, 1998.

[2] D. P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, arXiv:1412.6980, 2014.

[3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*, Journal of Machine Learning Research, 15:1929–1958, 2014.

[4] X. Glorot and Y. Bengio, *Understanding the difficulty of training deep feedforward networks*, AISTATS, pp. 249–256, 2010.

[5] V. Nair and G. E. Hinton, *Rectified Linear Units Improve Restricted Boltzmann Machines*, ICML, pp. 807–814, 2010.

[6] A. Abid et al., *Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild*, arXiv:1906.02569, 2019.